

Automated Debugging of Datalog Programs

Jiashen Wei* Baoyuan Luo* Runshuo Xie Yun Qi Yiyu Zhang Xizao Wang Xintao Niu# Zhiqiang Zuo#

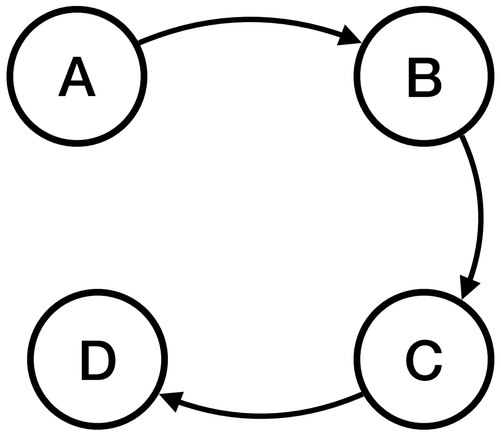
Speaker

*: equal contribution #: corresponding authors

Nanjing University, China

- Datalog is a declarative programming language
- Datalog programs consist of *rules* that operate on input *facts*
- Input facts describe what we know:

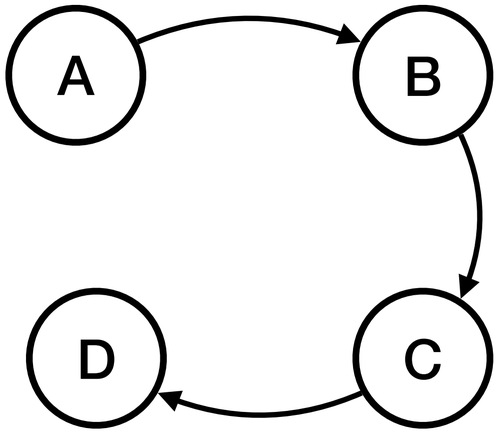
```
edge(a,b), edge(b,c), edge(c,d)
```



- Rules describe how to derive new facts:

```
path(X,Y) :- edge(X,Y).
path(X,Z) :- path(X,Y), edge(Y,Z).
```

- Conceptually, rules are repeatedly applied until reaching a fixpoint



r1

```
path(X,Y) :-
      edge(X,Y).
```

r2

```
path(X,Z) :-
      path(X,Y),
      edge(Y,Z).
```

When some derived facts are wrong, how can we find the faulty rules?

```
edge(a,b).
edge(b,c).
edge(c,d).
```

→

```
edge(a,b).
edge(b,c).
edge(c,d).
path(a,b).
path(b,c).
path(c,d).
```

→

```
edge(a,b).
edge(b,c).
edge(c,d).
path(a,b).
path(b,c).
path(c,d).
path(a,c).
path(b,d).
```

→

```
edge(a,b).
edge(b,c).
edge(c,d).
path(a,b).
path(b,c).
path(c,d).
path(a,c).
path(b,d).
path(a,d).
```

Reachability Analysis

```
class Main {
    static void main(String[] args) {
        init();

        Parser p = new JsonParser();
        p.parse();
    }

    static void init() {
        loadConfig();
    }

    static void loadConfig() {}
}
```

```
interface Parser {
    void parse();
}
```

```
class JsonParser
implements Parser {
    public void parse() {
        readJson();
    }
    void readJson() {}
}
```

```
class YamlParser
implements Parser {
    public void parse() {
        readYaml();
    }
    void readYaml() {}
}
```

Reachability Analysis

```
class Main {
    static void main(String[] args) {
        init();

        Parser p = new JsonParser();
        p.parse();
    }

    static void init() {
        loadConfig();
    }

    static void loadConfig() {}
}
```

```
interface Parser {
    void parse();
}
```

```
class JsonParser
implements Parser {
    public void parse() {
        readJson();
    }
    void readJson() {}
}
```

```
class YamlParser
implements Parser {
    public void parse() {
        readYaml();
    }
    void readYaml() {}
}
```

```
reachable(Main.main)
reachable(Main.init)
reachable(Main.loadConfig)
reachable(JsonParser.parse)
reachable(JsonParser.readJson)
reachable(YamlParser.parse)
reachable(YamlParser.readYaml)
```

Reachability Analysis

```
class Main {
    static void main(String[] args) {
        init();

        Parser p = new JsonParser();
        p.parse();
    }

    static void init() {
        loadConfig();
    }

    static void loadConfig() {}
}
```

```
interface Parser {
    void parse();
}
```

```
class JsonParser
implements Parser {
    public void parse() {
        readJson();
    }
    void readJson() {}
}
```

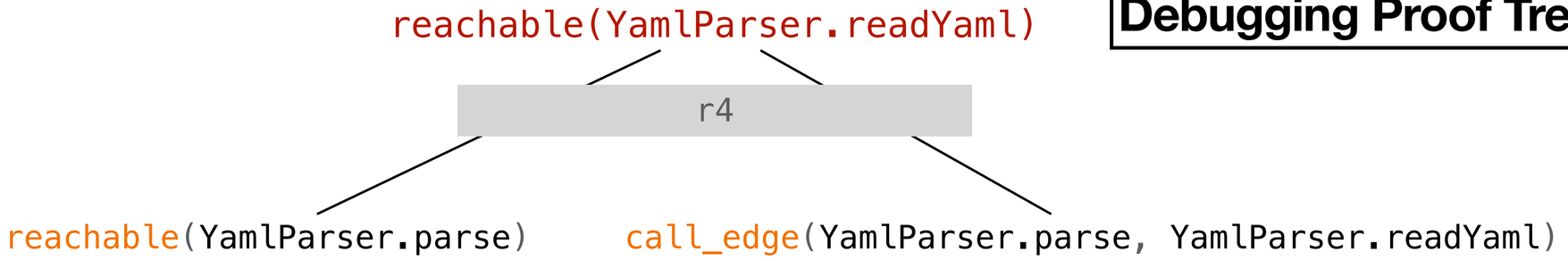
```
class YamlParser
implements Parser {
    public void parse() {
        readYaml();
    }
    void readYaml() {}
}
```

reachable(Main.main) ✓
reachable(Main.init) ✓
reachable(Main.loadConfig) ✓
reachable(JsonParser.parse) ✓
reachable(JsonParser.readJson) ✓
reachable(YamlParser.parse) ✗
reachable(YamlParser.readYaml) ✗

reachable(YamlParser.readYaml)

Debugging Proof Tree

Debugging Proof Tree



r4

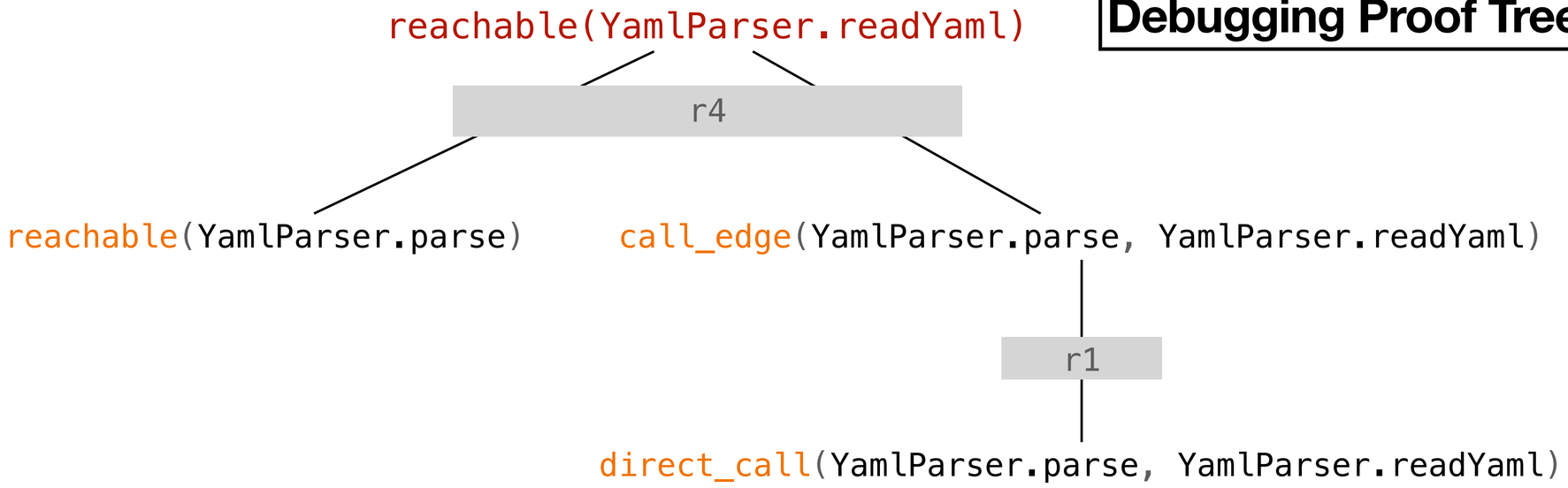
```
reachable(N) :-
      reachable(M),
      call_edge(M,N).
```

```
class YamlParser
implements Parser {
    public void parse() {
        readYaml();
    }
    void readYaml() {}
}
```

Debugging Proof Tree

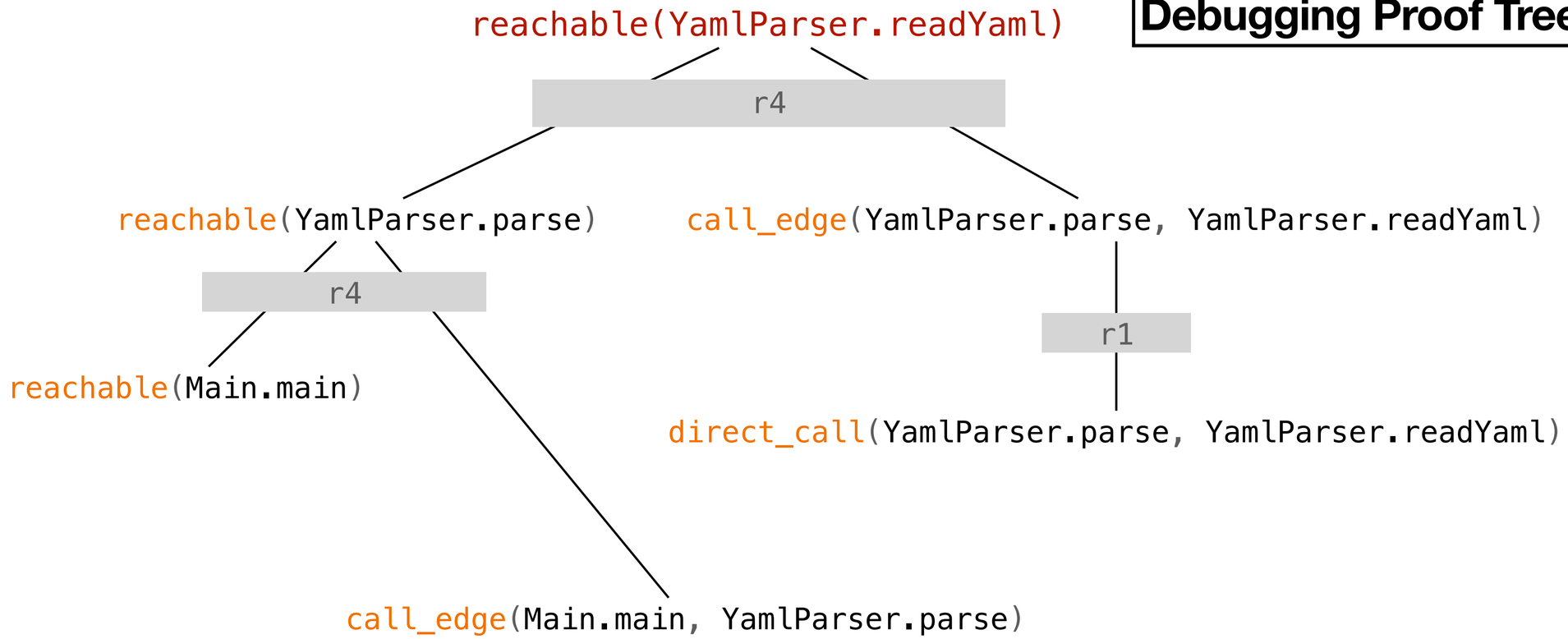
r1

```
call_edge(M, N) :-
      direct_call(M, N).
```




```
class YamlParser
implements Parser {
    public void parse() {
        readYaml();
    }
    void readYaml() {}
}
```

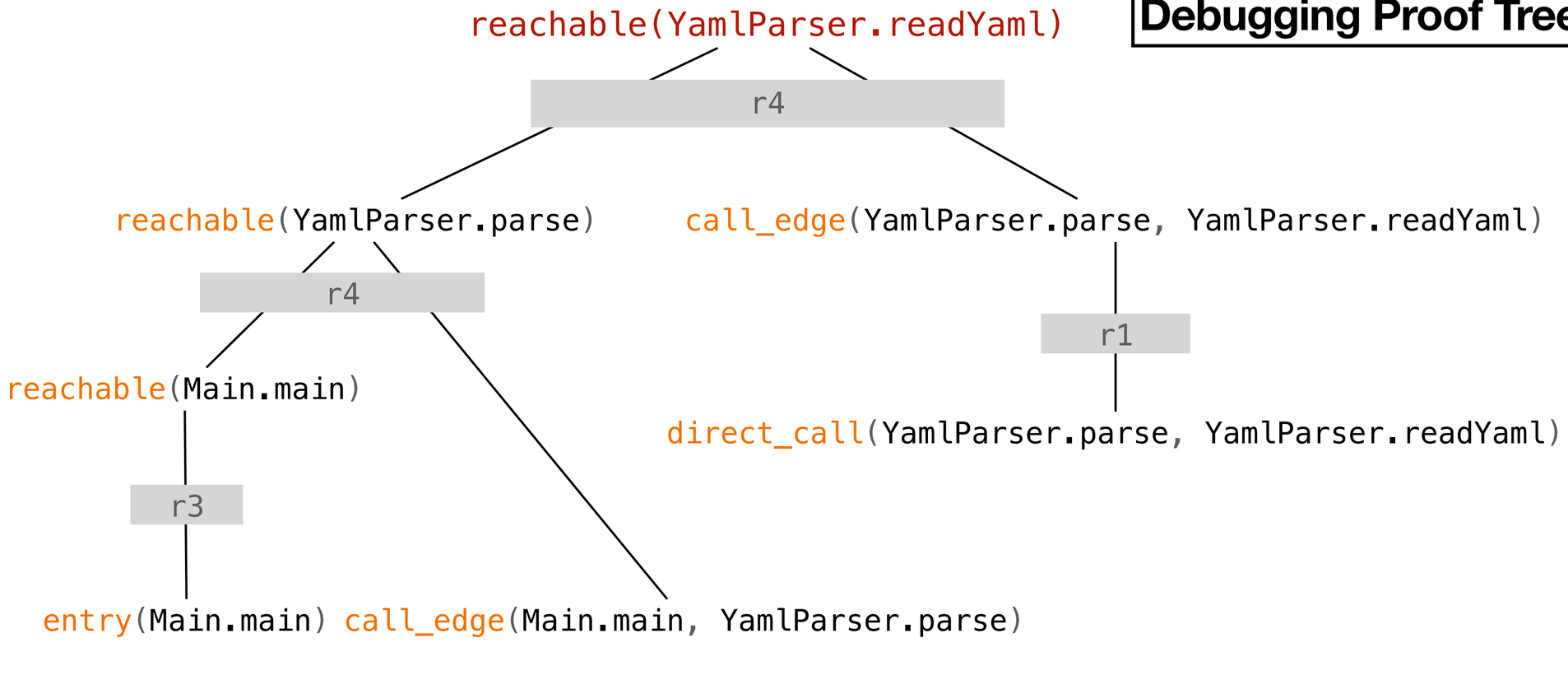
Debugging Proof Tree



r4

```
reachable(N) :-
      reachable(M),
      call_edge(M,N).
```

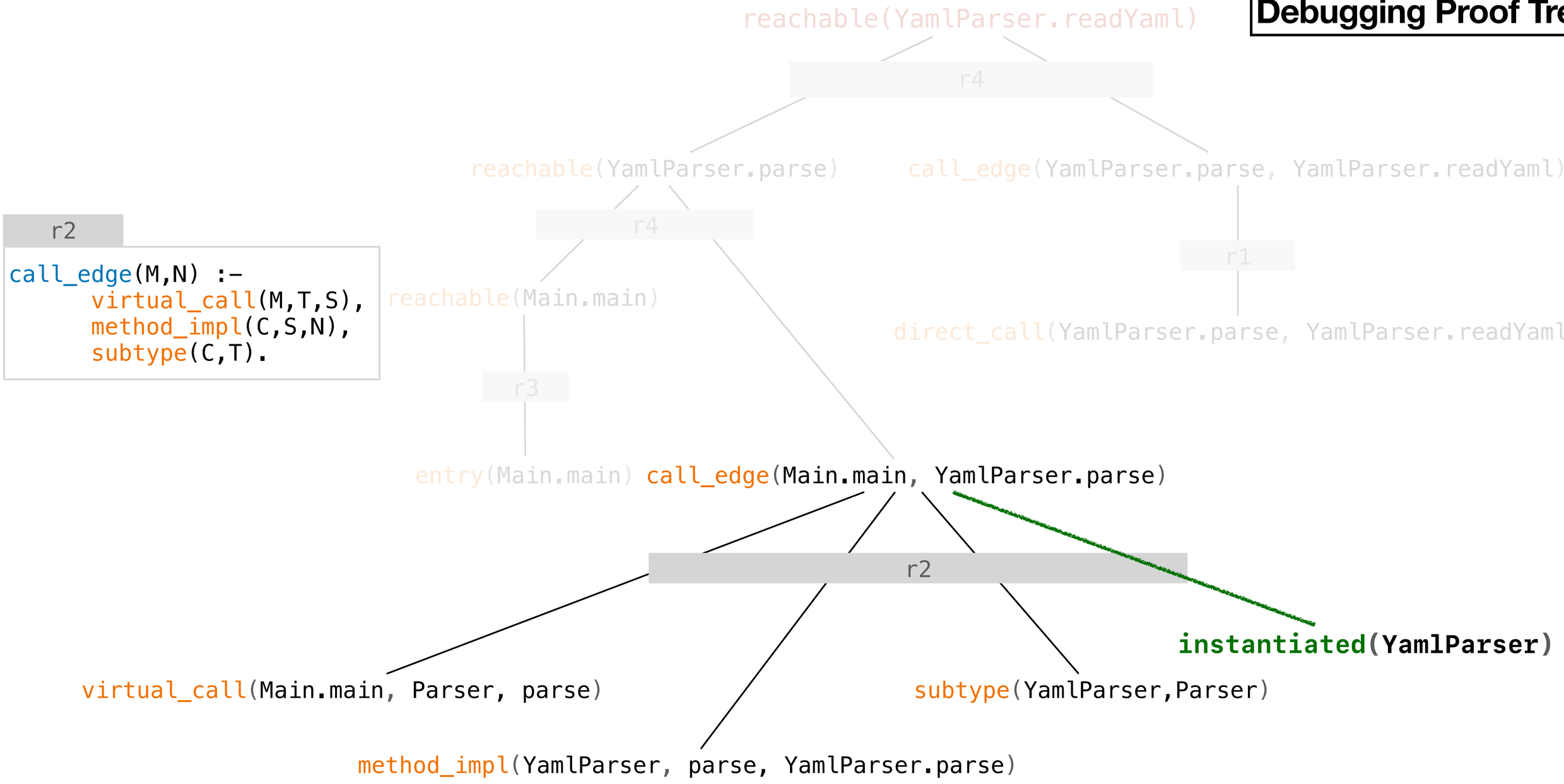
Debugging Proof Tree



r3

```
reachable(M) :-
      entry(M).
```

Debugging Proof Tree

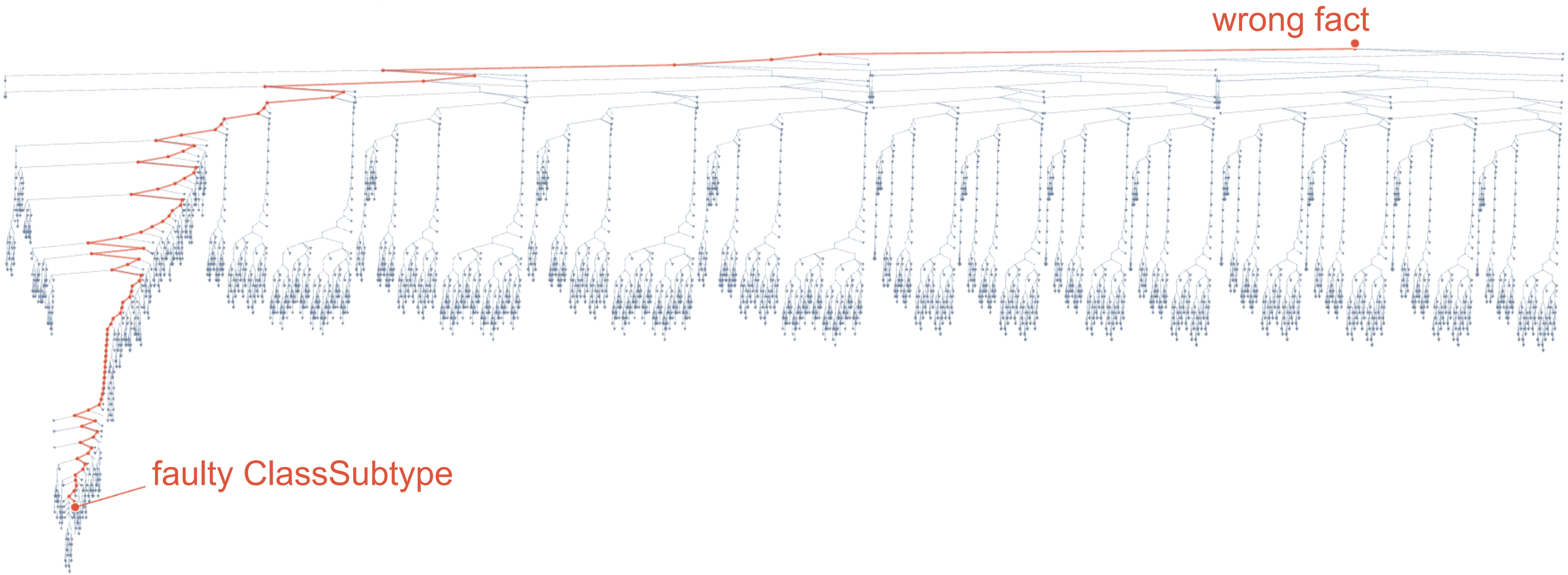


r2

```
call_edge(M,N) :-
      virtual_call(M,T,S),
      method_impl(C,S,N),
      subtype(C,T).
```

Fault #31

- It’s ok to debug a small proof tree
- How about a big one?



- 29,018 distinct facts
- 86 edges to the closest faulty rule

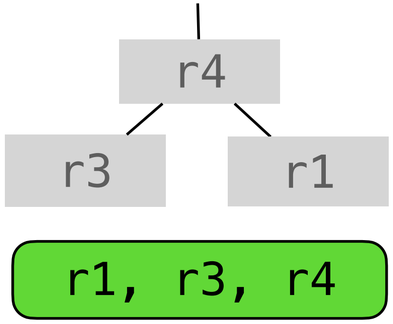
It’s So Hard

- It seems very hard to debug a fact
- Why not try to debug multiple facts at the same time?

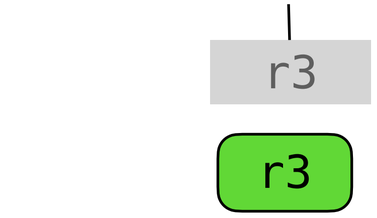


A New Perspective

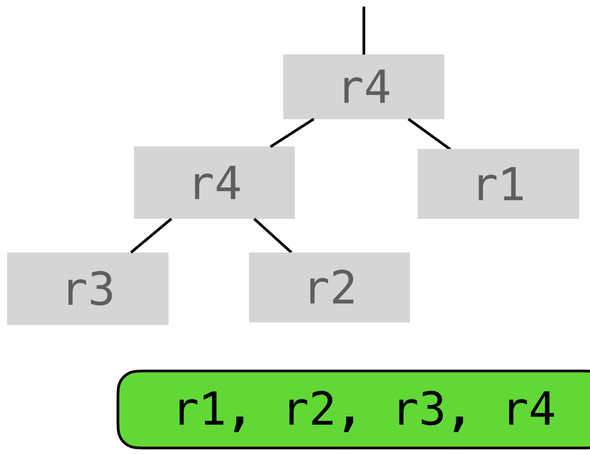
reachable(Main.init)



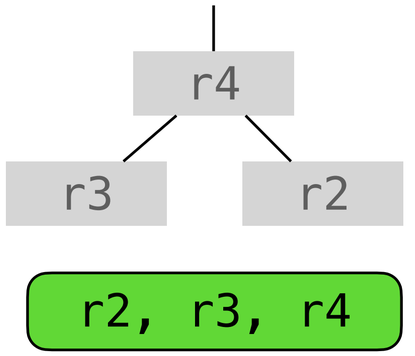
reachable(Main.main)



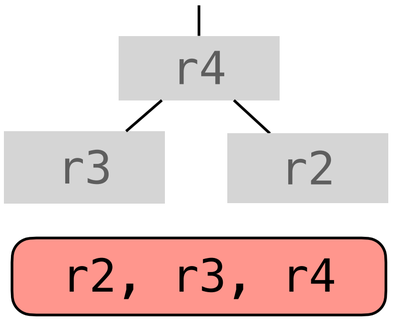
reachable(Main.loadConfig)



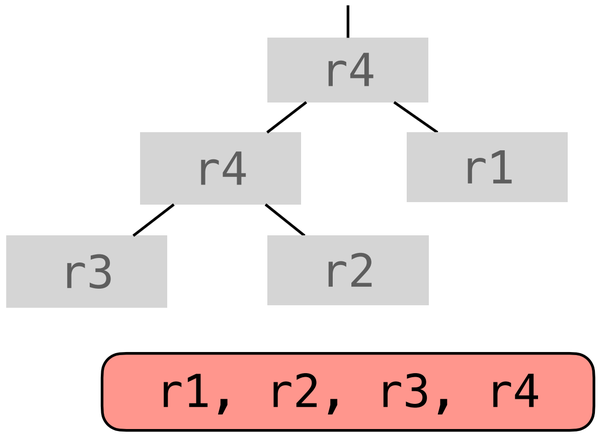
reachable(JsonParser.parse)



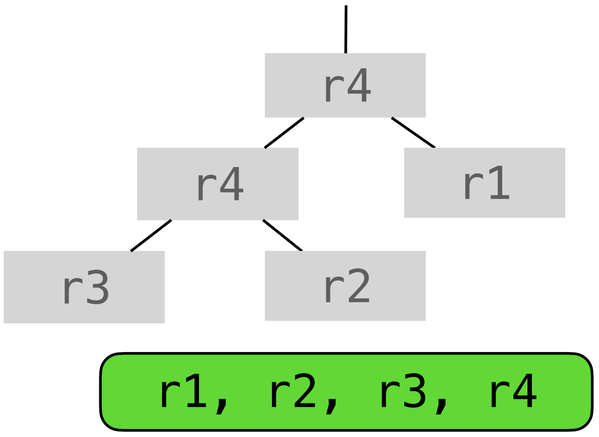
reachable(YamlParser.parse)



reachable(YamlParser.readYaml)



reachable(JsonParser.readJson)



Output fact	r1	r2	r3	r4
`reachable(Main.main)`			✓	
`reachable(Main.init)`	✓		✓	✓
`reachable(Main.loadConfig)`	✓		✓	✓
`reachable(JsonParser.parse)`		✓	✓	✓
`reachable(JsonParser.readJson)`	✓	✓	✓	✓
`reachable(YamlParser.parse)`		✓	✓	✓
`reachable(YamlParser.readYaml)`	✓	✓	✓	✓

Output fact	r1	r2	r3	r4
`reachable(Main.main)`			✓	
`reachable(Main.init)`	✓		✓	✓
`reachable(Main.loadConfig)`	✓		✓	✓
`reachable(JsonParser.parse)`		✓	✓	✓
`reachable(JsonParser.readJson)`	✓	✓	✓	✓
`reachable(YamlParser.parse)`		✓	✓	✓
`reachable(YamlParser.readYaml)`	✓	✓	✓	✓

- A faulty rule may tend to participate more in wrong facts than in correct ones.

SBFL

Spectrum-Based Fault Localization

```
1.int calculatePrice(User user) {
2.    int price = 100;
3.    if (user.isPremium())
4.        price -= 10;
5.    if (user.hasCoupon())
6.        price -= 20;
7.    if (user.isStudent()) {
8.        price += 30;   // should be -= 30
9.   }
10.    return price;
11.}
```

Program under testing

Testing

✓ t_1

In: Normal

Expected: 100

Out: 100

```
1.int calculatePrice(User user) {
2.     int price = 100;------------ L2
3.     if (user.isPremium())
4.         price -= 10;
5.     if (user.hasCoupon())
6.         price -= 20;
7.     if (user.isStudent()) {
8.         price += 30;   // should be -= 30
9.    }
10.     return price;
11.}
```

Program under testing

Testing

✓ t_2

In: Premium

Expected: 90

Out: 90

```
1.int calculatePrice(User user) {
2.     int price = 100;
3.     if (user.isPremium())
4.         price -= 10;
5.     if (user.hasCoupon())
6.         price -= 20;
7.     if (user.isStudent()) {
8.         price += 30;   // should be -= 30
9.    }
10.     return price;
11.}
```

L2

L4

Program under testing

Testing

✘ t_3

In: Student

Expected: 70

Out: 130

```
1.int calculatePrice(User user) {
2.    int price = 100;-----------------------------------L2
3.    if (user.isPremium())
4.        price -= 10;
5.    if (user.hasCoupon())
6.        price -= 20;
7.    if (user.isStudent()) {
8.        price += 30;   // should be -= 30 ----------------------L8
9.   }
10.    return price;
11.}
```

Program under testing

Testing

✘ t_4

In: Student with Premium and Coupon

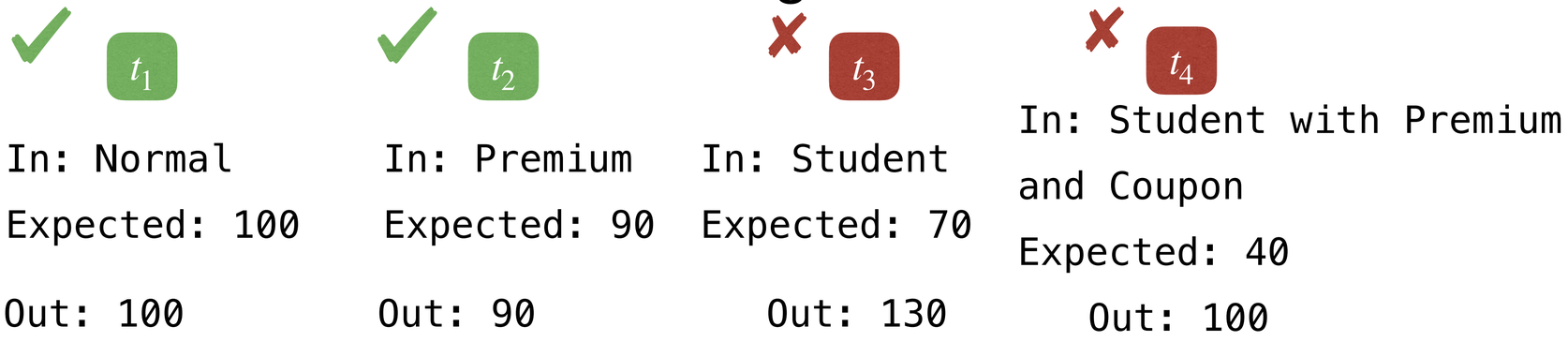
Expected: 40

Out: 100

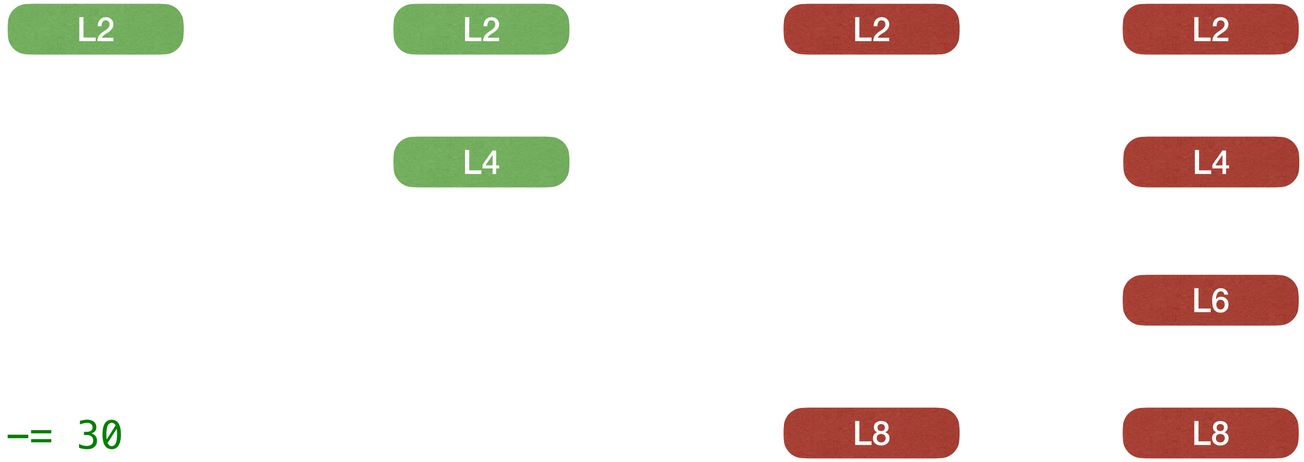
```
1.int calculatePrice(User user) {
2.    int price = 100;                                          L2
3.    if (user.isPremium())
4.        price -= 10;                                          L4
5.    if (user.hasCoupon())
6.        price -= 20;                                          L6
7.    if (user.isStudent()) {
8.        price += 30;   // should be -= 30                     L8
9.   }
10.    return price;
11.}
```

Program under testing

Testing




```
1.int calculatePrice(User user) {
2.     int price = 100;
3.     if (user.isPremium())
4.         price -= 10;
5.     if (user.hasCoupon())
6.         price -= 20;
7.     if (user.isStudent()) {
8.         price += 30;   // should be -= 30
9.    }
10.     return price;
11.}
```



```
1.int calculatePrice(User user) {
2.     int price = 100;
3.     if (user.isPremium())
4.         price -= 10;
5.     if (user.hasCoupon())
6.         price -= 20;
7.     if (user.isStudent()) {
8.         price += 30;   // should be -= 30
9.    }
10.     return price;
11.}
```

Test	L2	L4	L6	L8
t1	✓			
t2	✓	✓		
t3	✓			✓
t4	✓	✓	✓	✓

Suspiciousness Score

Test	L2	L4	L6	L8
t1	✓			
t2	✓	✓		
t3	✓			✓
t4	✓	✓	✓	✓
Suspiciousness score	0.707	0.500	0.707	**1.000**

Suspiciousness Score

Test	L2	L4	L6	L8
t1	✓			
t2	✓	✓		
t3	✓			✓
t4	✓	✓	✓	✓
Suspiciousness score	0.707	0.500	0.707	**1.000**

Output fact	r1	r2	r3	r4
`reachable(Main.main)`			✓	
`reachable(Main.init)`	✓		✓	✓
`reachable(Main.loadConfig)`	✓		✓	✓
`reachable(JsonParser.parse)`		✓	✓	✓
`reachable(JsonParser.readJson)`	✓	✓	✓	✓
`reachable(YamlParser.parse)`		✓	✓	✓
`reachable(YamlParser.readYaml)`	✓	✓	✓	✓

Suspiciousness Score

Test	L2	L4	L6	L8
`t1`	✓			
`t2`	✓	✓		
`t3`	✓			✓
`t4`	✓	✓	✓	✓
Suspiciousness score	0.707	0.500	0.707	**1.000**

Output fact	r1	r2	r3	r4
`reachable(Main.main)`			✓	
`reachable(Main.init)`	✓		✓	✓
`reachable(Main.loadConfig)`	✓		✓	✓
`reachable(JsonParser.parse)`		✓	✓	✓
`reachable(JsonParser.readJson)`	✓	✓	✓	✓
`reachable(YamlParser.parse)`		✓	✓	✓
`reachable(YamlParser.readYaml)`	✓	✓	✓	✓
Suspiciousness score	0.354	**0.707**	0.535	0.577

End of The Story?

- Could SBFL actually be a particularly good fit for Datalog?
  - More of an intuition

Same Coverage, Different Relevance

Test	L8	L9	L10
t1			
t2			
t3	✓	✓	✓
t4	✓	✓	✓

```
1.int calculatePrice(User user) {
2.     int price = 100;
3.     if (user.isPremium())
4.         price -= 10;
5.     if (user.hasCoupon())
6.         price -= 20;
7.     if (user.isStudent()) {
8.         price += 30;   // should be -= 30
9.         logStudentDiscount(user);
10.        recordPromotion(user);
11.    }
12.    return price;
13.}
```

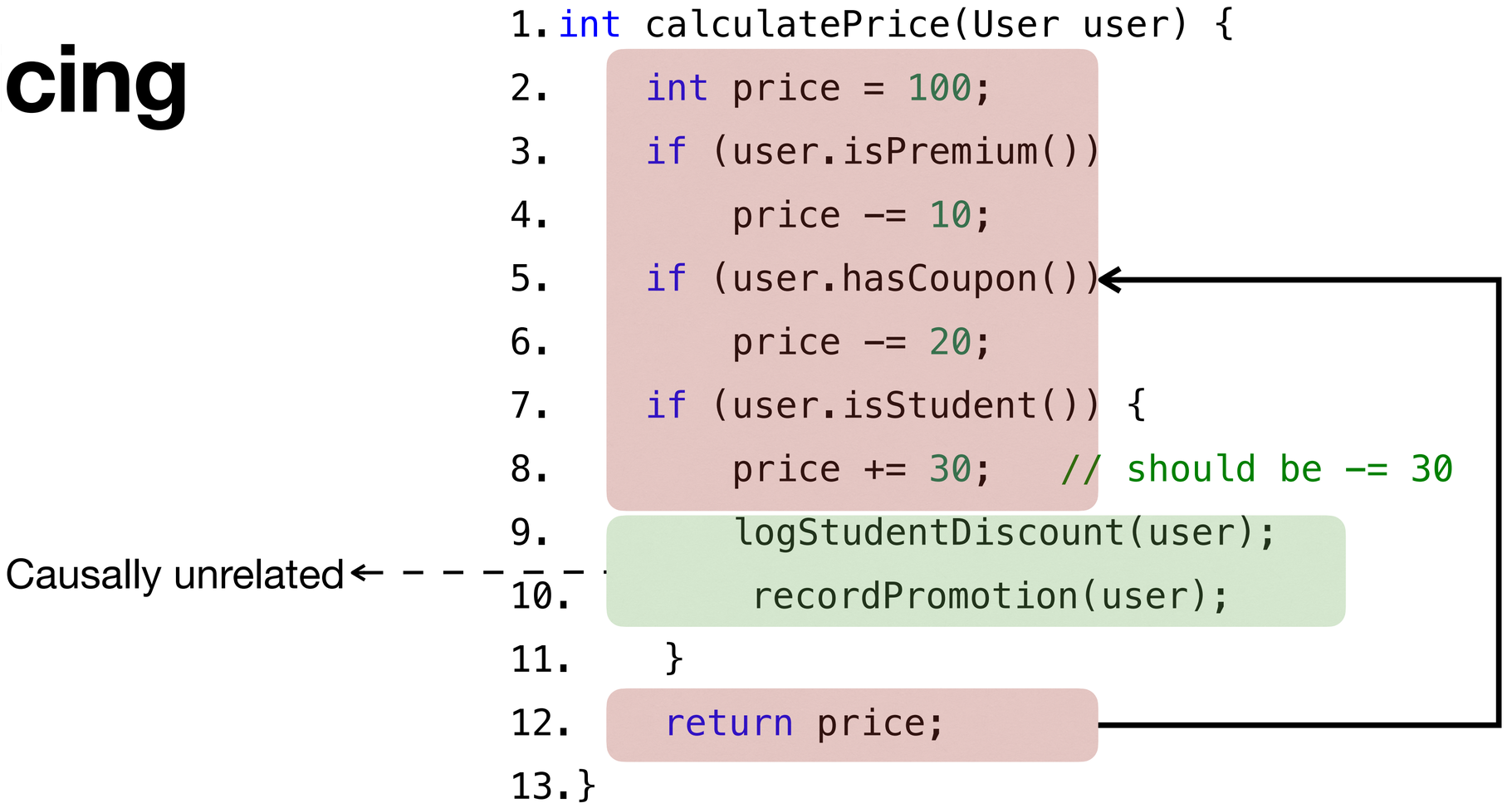
always occurs with failure

Slicing

```
1. int calculatePrice(User user) {
2.     int price = 100;
3.     if (user.isPremium())
4.         price -= 10;
5.     if (user.hasCoupon())
6.         price -= 20;
7.     if (user.isStudent()) {
8.         price += 30;   // should be -= 30
9.         logStudentDiscount(user);
10.        recordPromotion(user);
11.    }
12.    return price;
13.}
```

Causally unrelated ← - - - - - -

Slicing



- Datalog gives us a kind of built-in slicing

- How do we know if it actually works?
- We need real Datalog bugs with ground truth
  - But there was no standard benchmark for Datalog debugging

Doop

Large, real-world Datalog code with over a decade of development

`Fix precise-generics multi-phase analysis`

"Fix ..." in the commit message

master · doop-completeness ··· 4.20.40 — 1 parent 8bf5781 commit a73b6e8

4 files changed +88 -87

Filter files...

- souffle-logic
  - analyses/adaptive-2-object-...
    - analysis.dl
  - main
    - context-sensitivity-declara...
    - context-sensitivity.dl
  - src/main/groovy/org/clyze/d...
    - SouffleGenericsMultiPhaseA...

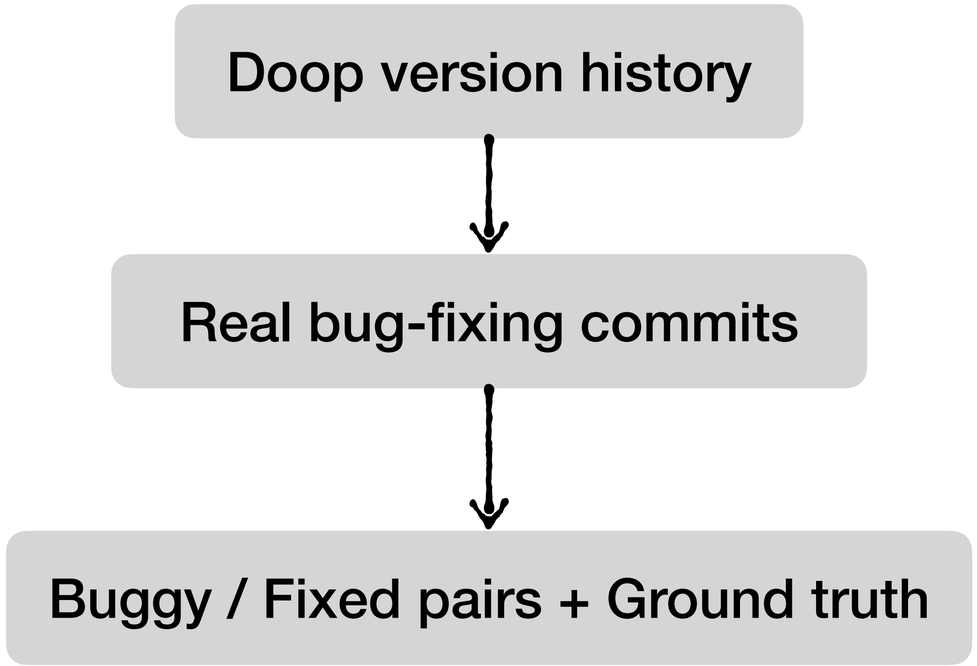
Search within code

souffle-logic/analyses/adaptive-2-object-sensitive+heap/analysis.dl +2 -1

souffle-logic/main/context-sensitivity-declarations.dl +21 -2

```
@@ -55,8 +55,8 @@
55  55
56  56   .decl OptStoreArrayIndex(?from:Var, ?base:Var)
57  57
58      - //.decl OptArgAssign(?toCtx:configuration.Context, ?to:Var, ?fromCtx:configuration.Context, ?from:Var, ?allowedType:Type)
59      - .decl OptArgAssign(?toCtx:configuration.Context, ?to:Var, ?fromCtx:configuration.Context, ?from:Var)
    58  + .decl OptArgAssign(?toCtx:configuration.Context, ?to:Var, ?fromCtx:configuration.Context, ?from:Var, ?allowedType:Type)
    59  + //.decl OptArgAssign(?toCtx:configuration.Context, ?to:Var, ?fromCtx:configuration.Context, ?from:Var)
60  60   #ifndef INFORMATION_FLOW
61  61   .decl OptReturnAssign(?toCtx:configuration.Context, ?to:Var, ?fromCtx:configuration.Context, ?from:Var)
62  62   #endif // INFORMATION_FLOW
@@ -99,3 +99,22 @@
99  99   // Is the ValueToRefine the real list of objects needing
100 100  // refinement or its negation?
```

- How do we know if it actually works?
- We need real Datalog bugs with ground truth
  - But there was no standard benchmark for Datalog debugging



- Finally:
  - 37 unique faults
  - 96 benchmark instances

- Benchmark Construction is not just version diff

- Benchmark Construction is not just version diff

Make old Doop versions runnable

- Broken dependencies
- Build / compatibility failures
- Network & repository issues

- Benchmark Construction is not just version diff

Make old Doop versions runnable

- Broken dependencies
- Build / compatibility failures
- Network & repository issues

Align executions semantically

- Different internal representations
- Different symbol tables / IDs
- Complex runtime representations

- Benchmark Construction is not just version diff

Make old Doop versions runnable

- Broken dependencies
- Build / compatibility failures
- Network & repository issues

Align executions semantically

- Different internal representations
- Different symbol tables / IDs
- Complex runtime representations

Reconstruct bugs & ground truth

- Difference ≠ bug
- Isolate fault-inducing changes
- Separate multiple independent faults

Evaluation

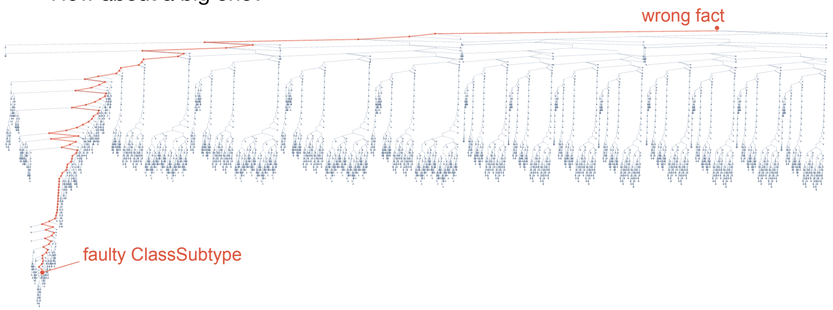
- Main results:
  - Rule localization works quite well.
  - With Ochiai, we achieve up to 87.5% Hit@1 (Top-1 hit rate)

Takeaways

Proof Tree too big

Fault #31

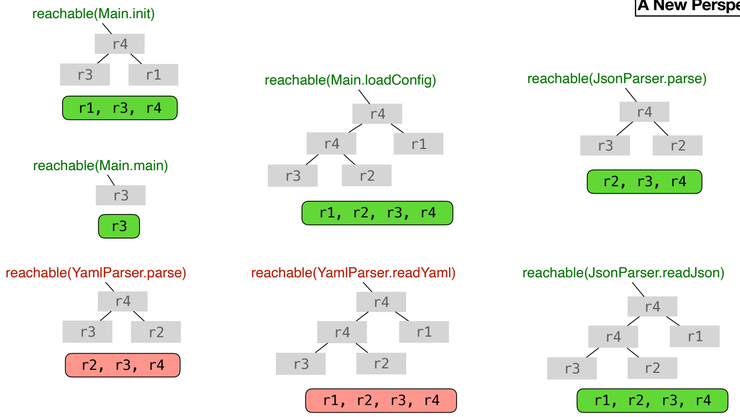
- It's ok to debug a small proof tree
- How about a big one?



- 29,018 distinct facts
- 86 edges to the closest faulty rule

We can leverage multiple trees

A New Perspective



SBFL is not that good for imperative programs

Same Coverage, Different Relevance

Test	L8	L9	L10
t1			
t2			
t3	✓	✓	✓
t4	✓	✓	✓

```
1.int calculatePrice(User user) {
2.    int price = 100;
3.    if (user.isPremium())
4.        price -= 10;
5.    if (user.hasCoupon())
6.        price -= 20;
7.    if (user.isStudent()) {
8.        price += 30;   // should be -= 30
9.        logStudentDiscount(user);
10.       recordPromotion(user);
11.   }
12.   return price;
13.}
```

always occurs with failure

SBFL could be better for datalog programs

A New Perspective

Output fact	r1	r2	r3	r4
reachable(Main.main)			✓	
reachable(Main.init)	✓		✓	✓
reachable(Main.loadConfig)	✓		✓	✓
reachable(JsonParser.parse)		✓	✓	✓
reachable(JsonParser.readJson)	✓	✓	✓	✓
reachable(YamlParser.parse)		✓	✓	✓
reachable(YamlParser.readYaml)	✓	✓	✓	✓

- A faulty rule may tend to participate more in wrong facts than in correct ones.

Thanks For Listening

Jiashen Wei* Baoyuan Luo* Benchmark Construction, Implementation and Evaluation

Xintao Niu# Advising

Runshuo Xie Help with the writing and presentation

Yun Qi Yiyu Zhang Xizao Wang Early Exploration on toy programs

Zhiqiang Zuo# Proposing this idea, initial draft